Advanced Alignment, Post-Training, and Synthetic Data

Preference Objectives, Reward Design, Oversight, Synthetic Data, and Model Merging

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 11, 2026

Table of Contents

1. Motivation
2. Learning Outcomes
3. Post-Training Objectives: From Preferences to Verifiable Rewards
4. The Reward Signal: AI Feedback, Reward Shaping and Reward Hacking
5. Specifications and Scalable Oversight
6. Synthetic Data Generation Pipelines
7. Data Curation at Foundation-Model Scale
8. Self-Improving Language Models
9. Continual Post-Training and Model Merging
10. Open-Source Post-Training Pipelines
11. Summary

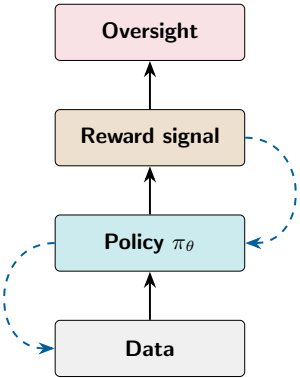
Post-Training Moved. The Bottleneck Moved with It.

- In 2023 and 2024 the centre of post-training was **RLHF and DPO** on human preferences.
- By 2025 and 2026 the open recipe is SFT, then an optional DPO-style stage, then large-scale on-policy RL with verifiable rewards.
- The scarce resource is no longer the optimiser. It is the **signal**:
  - **Cost.** A human preference label costs 5 to 20 dollars. An AI label costs under one cent.
  - **Data.** More than 98% of the alignment data of Nemotron-4 340B is synthetic.
  - **Compute.** DeepSeek-V3.2 reports post-training compute above 10% of its pre-training cost.
- Once humans leave the loop, who checks the reward, the data and the model?

Every oversight signal degrades when you optimise against it: a reward model, a judge, a chain-of-thought monitor, a self-generated label.

Lambert, "The state of post-training in 2025" (Interconnects, 2025). NVIDIA, "Nemotron-4 340B Technical Report," arXiv:2406.11704. DeepSeek-AI, "DeepSeek-V3.2," arXiv:2512.02556.

Today's Map: One Loop, Seven Questions



3. Who oversees a stronger model?
Specifications, debate, critics

2. Where does the reward come from?
AI feedback, shaping, hacking

1. Which objective, which optimiser?
DPO family, GRPO family

4. Where does the data come from?
Synthetic pipelines, curation

5. Can the loop close on itself?
Self-improving language models

6. Can it run for years?
Continual post-training, merging

7. What does it look like in public?
Tülu 3, Olmo 3, RL frameworks

Learning Outcomes

By the end of this session, you should be able to:

- **Place** IPO, SimPO, length-normalised DPO and GRPO variants on one map, with the failure each fixes.
- **Explain** why frontier recipes moved from offline preferences to on-policy RL with verifiers.
- **Design** a reward for RL post-training, and predict how it will be over-optimised or hacked.
- **Compare** oversight protocols: debate, weak-to-strong generalisation, critics and specifications.
- **Build** a synthetic data pipeline with the right quality controls, and state when it collapses.
- **Analyse** self-improvement loops: what bounds them and how their evaluations mislead.
- **Plan** continual updates, merging and an open-source post-training run end to end.

Prerequisite knowledge: the RLHF pipeline, Bradley–Terry reward models, reward over-optimisation; PPO with a value model; DPO, ORPO, KTO, and DPO as offline optimisation; GRPO, GSPO, RL with verifiable rewards, DeepSeek-R1; process and outcome reward models; entropy collapse in RL; Constitutional AI, RLAIF; LLM-as-a-judge biases; knowledge and reasoning-trace distillation; benchmark contamination detection; Constitutional Classifiers; fine-tuning attacks on safety training; catastrophic forgetting, replay, EWC, LoRA; continued pre-training with re-warming and replay; task vectors, TIES and DARE merging.

Alignment and Post-Training: **Objectives and Algorithms**

What replaced RLHF and DPO at the frontier, and why?

One Map for Every Objective

- DeepSeekMath's *unified paradigm*: every method has the same gradient, and differs in three choices.

$$\nabla_\theta \mathcal{J}_\mathcal{A}(\theta) = \mathbb{E}_{(q,o)\sim\mathcal{D}}\left[\frac{1}{|o|}\sum_t GC_\mathcal{A}(q,o,t,\pi_{rf})\,\nabla_\theta \log \pi_\theta(o_t|q,o_{<t})\right]$$

Method	Data source	On-policy	Negative gradient
SFT, offline best-of-N	fixed dataset, $GC = 1$	✗	✗
DPO, IPO and variants	fixed pairs	✗	✓
ReST, online best-of-N	live policy, filtered	✓	✗
REINFORCE, PPO, GRPO	live policy, any reward	✓	✓

- Both axes do the same job: they make the update **mode-seeking**. If the reward peak lies where π_{ref} puts little mass, you need on-policy samples to find it.

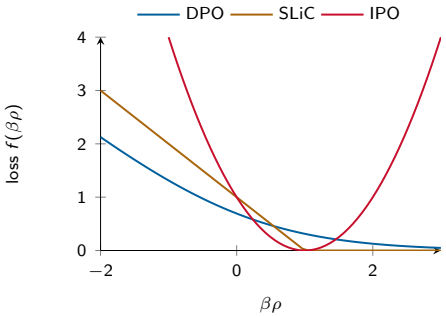
- DeepSeekMath's own finding: online data beats offline data, with everything else fixed.

Shao et al., "DeepSeekMath," arXiv:2402.03300, Eq. 5 and Table 10. Tajwar et al., "Preference Fine-Tuning of LLMs Should Leverage Suboptimal, On-Policy Data," ICML 2024, arXiv:2404.14367, Table 1 (rows follow the paper's prose).

Offline Losses Are One Family: DPO, SLiC, IPO

- Let $\rho = \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}$, the log-ratio gap between winner and loser. Each offline loss is $f(\beta\rho)$ for a convex f, a surrogate of the 0-1 loss.
- **Why IPO exists.** A preference observed once looks deterministic. Under Bradley-Terry that pushes $r(y_w) - r(y_l) \to \infty$, so $\pi(y_l) \to 0$ *whatever* the regularisation strength.
- **IPO** regresses the gap onto a finite target set by the regularisation strength τ:

$$\mathcal{L}_{\text{IPO}} = \mathbb{E}\Big[\big(\rho - \tfrac{1}{2\tau}\big)^2\Big]$$



IPO's evidence is a 3-action bandit. The paper has **no LLM experiments**.

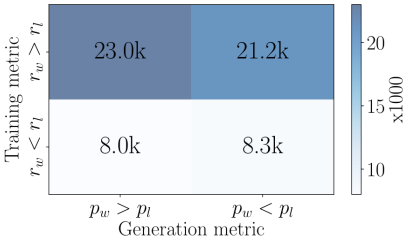
Tang et al., "Generalized Preference Optimization," arXiv:2402.05749, Table 1. Azar et al., "A General Theoretical Paradigm to Understand Learning from Human Preferences," arXiv:2310.12036, Sec. 4.2 and Eq. 17.

SimPO: Train on What You Decode With

- **The mismatch.** DPO ranks by a log-ratio to π_{ref}. Decoding ranks by likelihood. Only about half of DPO training triples agree on both.
- **SimPO** drops the reference model and uses the length-normalised log-likelihood as the reward, with a target margin γ:

$$\mathcal{L} = -\mathbb{E}\left[\log \sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x) - \frac{\beta}{|y_l|}\log \pi_\theta(y_l|x) - \gamma\right)\right]$$

- Length normalisation matters: the correlation between likelihood and length falls from 0.82 to 0.34.
- About 10% less memory and 20% less run time than DPO.



DPO training pairs: reward ranking (rows) against likelihood ranking (columns).

Meng, Xia, Chen, "SimPO: Simple Preference Optimization with a Reference-Free Reward," arXiv:2405.14734, Figures 4b and 5c, Table 6.

What Survived: Length-Normalised DPO

Tülu 3 ablation	Average
SFT checkpoint	55.7
SimPO	51.8, 52.9
DPO	55.2
PPO	54.5 to 55.5
DPO, length-norm.	57.3

- SimPO's headline gains did not transfer. In Tülu 3, *only* length-normalised DPO beat the SFT checkpoint.
- What the field kept is one idea from SimPO's appendix: **divide each log-ratio by its length**, and keep π_{ref}.

Other variants, by the failure each one patches:

Label noise	Likelihood displacement	Static β	Distribution shift
cDPO, Robust DPO	DPOP, RPO, CPO	β-DPO	iterative and online DPO

Lambert's verdict: "the choice of algorithm is far less important than the initial model and the data used".

Lambert et al., "Tülu 3," arXiv:2411.15124, Table 18. SimPO repository. Hugging Face TRL paper index. Lambert, RLHF Book, "Direct-Alignment Algorithms".

Offline or Online? The Quantified Evidence

Benchmark	SFT	DPO	Iterative DPO	PPO
CodeContests, 10@1k (%)	15.2	0.0	3.2	**22.4**
APPS introductory, pass@5 (%)	38.6	n/a	34.2	**44.4**
SafeRLHF helpfulness reward	n/a	−2.86	−2.96	**+1.69**
SafeRLHF safety rate (%)	n/a	95.8	99.9	99.5

- **The counterpoint is cost.** In Tülu 3, PPO took about 28 hours on two nodes against 4 hours on one for DPO, for similar scores.

A spectrum, not a binary: offline DPO, iterative DPO, online DPO, PPO and GRPO. On-policy pays most when the reward is **accurate** and the task is **hard**.

Xu et al., "Is DPO Superior to PPO for LLM Alignment?" ICML 2024, arXiv:2404.10719, Tables 6 to 8. Guo et al., "Direct Language Model Alignment from Online AI Feedback," arXiv:2402.04792. Tülu 3, arXiv:2411.15124.

The GRPO Objective and Its Three Questioned Terms

- Every fix in the rest of this section starts from the same objective:

$$\mathcal{J} = \mathbb{E}\Big[\frac{1}{G}\sum_i \frac{1}{|o_i|}\sum_t \min\left(r_{i,t}\hat{A}_{i,t},\ \mathrm{clip}(r_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}\right)\Big] - \beta D_{KL}, \qquad \hat{A}_{i,t} = \frac{R_i - \mathrm{mean}(\{R_j\})}{\mathrm{std}(\{R_j\})}$$

- G responses o_i per prompt, with rewards R_i. $r_{i,t}$ is the token probability ratio to the policy that sampled.
- Three terms have since been **questioned**: $1/|o_i|$, the division by std, and the KL term.

Objective as written in "DAPO," arXiv:2503.14476, Eqs. 8 to 9, after Shao et al., arXiv:2402.03300.

GRPO's Hidden Biases (Dr. GRPO)

Length bias, from $1/|o_i|$

- Short correct answers get larger updates.
- Long *wrong* answers are penalised less. So wrong answers grow longer during training.
- Not specific to GRPO: the PPO losses in TRL, OpenRLHF and verl all normalised by length.

Difficulty bias, from dividing by std

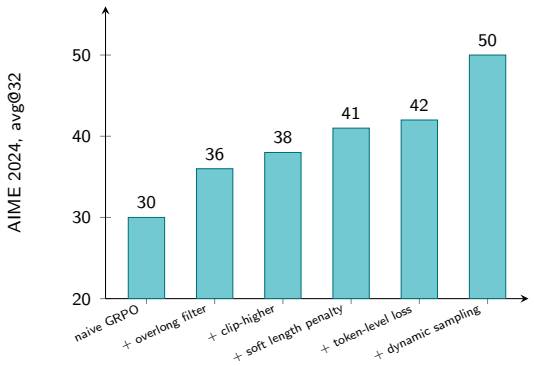
- Prompts that are almost always solved, or almost never, have a small std.
- Their advantages are inflated, so the easiest and hardest questions dominate.
- The std is "not a valid baseline". It breaks unbiasedness.

Fix: $\tilde{A}_i = R_i - \mathrm{mean}(\{R_j\})$, sum over tokens, divide by a constant

- Result: a 7B model reaches 43.3% on AIME 2024 in 27 hours on 8 A100 GPUs.

Liu et al., "Understanding R1-Zero-Like Training: A Critical Perspective," COLM 2025, arXiv:2503.20783, Sec. 3.1 and Table 2. Hashimoto, Stanford CS336 Lecture 16 (2025).

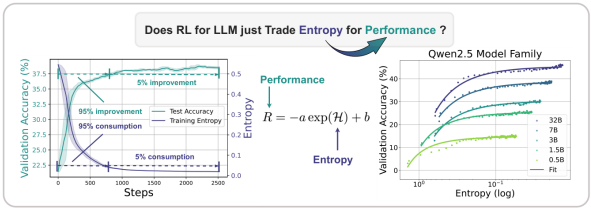
DAPO: Four Fixes, Twenty Points



- **Clip-higher.** Asymmetric clip, $\varepsilon_{low} = 0.2$ and $\varepsilon_{high} = 0.28$, to counter early entropy collapse.
- **Dynamic sampling.** Drop prompts whose group is all correct or all wrong. They carry no gradient.
- **Token-level loss.** Average over all tokens of the batch, not per sample.
- **Overlong shaping.** Mask or softly punish truncated responses.
- **No KL term.** A long-reasoning model must move far from its start.

ByteDance Seed and Tsinghua AIR, "DAPO: An Open-Source LLM Reinforcement Learning System at Scale," arXiv:2503.14476, Table 1 (Qwen2.5-32B base). The final 50 beats DeepSeek-R1-Zero-Qwen-32B (47) with half the steps.

Does RL Just Trade Entropy for Performance?



- An empirical law across 11 models: $R = -a\,e^{H} + b$. Reward rises only as policy entropy H falls, so the ceiling is fixed at $H = 0$.
- 73% of the entropy drop and 76% of the gain happen in the first 200 steps, one twelfth of training.
- **Fixes that keep exploration alive.** Clip-Cov and KL-Cov restrain only high-covariance tokens. CISPO clips the importance weight, not the update, so rare "fork" tokens such as *However* keep their gradient.

Cui et al., "The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models," arXiv:2505.22617, Figures 1 and 2. MiniMax, "MiniMax-M1," arXiv:2506.13585.

The Policy-Gradient Family on One Table

Method	Importance sampling	Clipping	Advantage or baseline
REINFORCE	none	none	Monte Carlo baseline
RLOO	none	none	leave-one-out
PPO	token level	two-sided	learned value function
GRPO	token level	two-sided	group-relative
GSPO	sequence level	two-sided	group-relative
CISPO	token level	stop-gradient on the weight	group-relative

- **Two rollouts can be enough.** GRPO with $G = 2$ is a contrastive, DPO-like objective. It keeps 97.6% of the performance of $G = 16$ with 12.5% of the rollouts.
- **Normalise over the batch, not the prompt.** With per-prompt normalisation, one GRPO run scored 95.0 on its AIME-24 training set and 0.0 on AIME-25.
- **The critic is not dead.** VAPO repairs value-based PPO with value pre-training and a length-adaptive GAE: 5 to 60 on AIME 2024.

Table: Lambert, RLHF Book, "Reinforcement Learning". Wu et al., "It Takes Two: Your GRPO Is Secretly DPO," arXiv:2510.00977. Hu et al., "REINFORCE++," arXiv:2501.03262. ByteDance Seed, "VAPO," arXiv:2504.05118.

The Sampler Is Not the Learner

- Rollouts come from an inference engine. Gradients come from a training framework. Their token probabilities **differ even when the weights are identical**.

Source of mismatch	**Fix reported**
Numerical precision of logits	compute the LM head in FP32: asymptotic reward 0.52 to 0.61 (ScaleRL)
Engine and trainer log-probs	truncated importance sampling on the ratio between them (Olmo 3)
MoE routing differs	*Keep Routing*: replay the inference-time routes in training (DeepSeek-V3.2)
Top-p and top-k truncation	*Keep Sampling Mask*: apply the same mask in training (DeepSeek-V3.2)

- ScaleRL, after 400,000 GPU-hours of ablations: most tricks change *efficiency*. Few move the *asymptote*.

Khatri et al., "The Art of Scaling Reinforcement Learning Compute for LLMs," arXiv:2510.13786. DeepSeek-AI, "DeepSeek-V3.2," arXiv:2512.02556. Olmo Team, "Olmo 3," arXiv:2512.13961.

What Labs Report Shipping

Model	Reported post-training recipe
Tülu 3 (Nov 2024)	SFT, length-normalised DPO on 354,192 on-policy pairs, then RLVR with PPO
Llama 4 (Apr 2025)	light SFT, online RL on medium and hard prompts, light DPO for corner cases
Qwen3 (May 2025)	GRPO on 3,995 query-verifier pairs. Small models by on-policy distillation
Kimi K2 (Jul 2025)	a squared-loss policy objective, with a self-critique rubric reward
GLM-4.5 (Aug 2025)	GRPO "excluding the KL loss term", with a two-stage difficulty curriculum
DeepSeek-V3.2 (Dec 2025)	GRPO with unbiased KL, off-policy masking, Keep Routing and Keep Sampling Mask
Olmo 3 (Dec 2025)	SFT, DPO with delta learning, then GRPO with no KL, no std, token-level loss, clip-higher

- **The trend:** the KL-to-reference anchor that defined RLHF and DPO is largely gone from RL recipes with verifiable rewards.
- **DPO changed jobs.** It went from *the* alignment algorithm to a cheap stage before or after RL.

Technical reports of each model: arXiv:2411.15124, 2505.09388, 2507.20534, 2508.06471, 2512.02556, 2512.13961. Meta AI, "The Llama 4 herd" (2025). Lambert, RLHF Book.

Cheaper Than RL: On-Policy Distillation

- If a strong teacher exists, do you need a reward at all?
- **On-policy distillation.** The *student* samples. The teacher scores every token. Minimise the per-token reverse KL on the states the student visits:

$$\mathrm{KL}\big(\pi_\theta \,\|\, \pi_{\text{teacher}}\big)$$

- **Why on-policy.** Offline imitation compounds error as $O(\epsilon L^2)$ over a length-L output. On-policy correction keeps it at $O(\epsilon L)$.
- Distillation also raised pass@64. RL did not.
- 2026: multi-teacher distillation (MOPD) beats mixed RL, cascaded RL and weight merging.

Qwen3-8B	AIME'24	GPU-h
RL	67.6	17,920
On-policy distillation	74.4	1,800

About one tenth of the compute, and a higher score.

Qwen Team, "Qwen3 Technical Report," arXiv:2505.09388, Table 21. Lu, "On-Policy Distillation" (Thinking Machines, 2025). Lambert, RLHF Book, "Synthetic Data and Distillation". Ma et al., "MOPD," arXiv:2606.30406.

Objectives: What to Remember

Symptom in an RL run	Fix
Wrong answers grow longer	drop $1/\lvert o\rvert$: Dr. GRPO, token-level loss
Entropy collapses early	clip-higher, CISPO, Clip-Cov and KL-Cov
Groups all correct or all wrong	dynamic or active sampling
Divergence with a fast engine or MoE	importance correction, FP32 head, Keep Routing

- Offline preference losses are one family. **Length-normalised DPO** is the variant that lasted.
- The 2026 consensus for RL: GRPO **minus KL, minus std**, plus token-level loss, clip-higher and sample filtering.

Next. Every method here assumed a reward R worth maximising. Where does R come from, and what happens when the model finds its flaws?

Diagnostic map compiled from the sources of this section. Wolfe, "GRPO++: Tricks for Making RL Actually Work" (2026).

Alignment and Post-Training: **The Reward Signal**

The optimiser is only as good as its reward. Where does the reward come from, and how does it break?

AI Feedback in 2026: Cheap, Consistent, Biased

	Human feedback	AI feedback
Cost per preference	5 to 20 dollars	under 0.01 dollar
Error profile	high noise, low bias	low noise, high bias

- **How good is the judge?** In UltraFeedback, GPT-4 agreed with a human annotator only 59.7% of the time, and 68.6% against the human majority.
- **A trained reward model can beat a prompted judge.** On hard chat comparisons, Nemotron-4's reward model reached 0.87 accuracy against 0.54 for an LLM judge.
- **Preference leakage.** When the data generator and the judge are related models, the judge favours the students of its relative. This bias is hard to detect.
- **Where it is used now.** As the reward for tasks with no verifier: a self-critique rubric in Kimi K2, an LM-judge score in $[0, 1]$ for chat in Olmo 3.

Lambert, "The state of post-training in 2025" (Interconnects, 2025), and RLHF Book, "Synthetic Data and Distillation". Cui et al., "UltraFeedback," arXiv:2310.01377, Table 4. NVIDIA, arXiv:2406.11704. Li et al., "Preference Leakage," ICLR 2026, arXiv:2502.01534.

Reward Shaping for Verifiable Rewards

- A verifier gives one bit. Shaping adds the terms that keep the run usable: format, length and language.

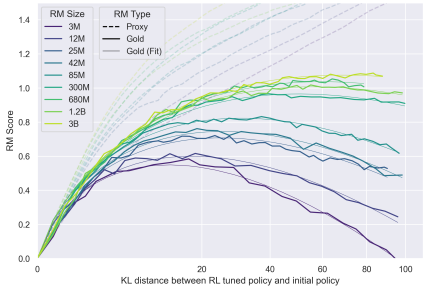
Recipe	Reward
Tülu 3	$\alpha = 10$ if the output verifies, else 0, minus a KL penalty
DeepSeek-R1	accuracy plus a format reward for the think tags. Later a language-consistency term
Magistral	format 0.1, correctness 0.9, language 0.1, length penalty up to -0.1

DAPO, overlong:
$$R_{\text{length}}(y) = \begin{cases} 0, & |y| \le L_{max} - L_{cache} \\ \dfrac{(L_{max} - L_{cache}) - |y|}{L_{cache}}, & L_{max} - L_{cache} < |y| \le L_{max} \\ -1, & L_{max} < |y| \end{cases}$$

- In Tülu 3, adding a learned reward model's score to the verifiable reward only added noise.

Tülu 3, arXiv:2411.15124, Eqs. 7 to 8. DeepSeek-R1, arXiv:2501.12948. Mistral AI, "Magistral," arXiv:2506.10910. DAPO, arXiv:2503.14476, Eq. 13.

When the Reward Is Learned: Over-Optimisation Has a Law



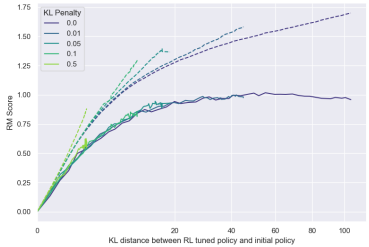
- **Setup.** A 6B "gold" reward model stands in for humans. Proxy models from 3M to 3B parameters are trained on its labels, then optimised against.
- **Result.** The proxy score (dashed) keeps rising. The gold score (solid) rises, peaks, then falls.
- With $d = \sqrt{D_{KL}(\pi \,\|\, \pi_{\text{init}})}$ the gold score follows

$$R_{\text{RL}}(d) = d\,(\alpha_{\text{RL}} - \beta_{\text{RL}} \log d)$$

- Larger reward models over-optimise less. Larger policies gain less, but do not over-optimise more.

Gao, Schulman, Hilton, "Scaling Laws for Reward Model Overoptimization," arXiv:2210.10760, Figures 1b and 3.

KL or No KL?



Gold score against KL, for five KL penalties.

- A KL penalty does **not** move the frontier. All penalties trace the same curve, and a larger one only stops earlier. **KL is early stopping.**
- What does help with a learned reward: conservative **ensembles** of reward models. In best-of-N they "practically eliminate" over-optimisation.
- **With a verifier there is no reward model to exploit.** DAPO and Magistral drop the KL term. Magistral: "the policy diverges substantially regardless".

This resolves a puzzle from the last section. Recipes dropped KL because the *reward* changed, not because the regulariser was wrong.

Gao et al., arXiv:2210.10760, Figure 9. Coste et al., "Reward Model Ensembles Help Mitigate Overoptimization," ICLR 2024, arXiv:2310.02743. Magistral, arXiv:2506.10910.

Reward Hacking Is Now a Production Problem

Case	Signal optimised	What happened
RLHF with time-limited human raters	human approval	raters' false-positive rate rose by 24.1% (QuALITY) and 18.3% (APPS)
GPT-4o rollback (May 2025)	user thumbs up and down, added to the reward	it "weakened the influence of our primary reward signal, which had been holding sycophancy in check"
o3 on agentic tasks (METR, Jun 2025)	task score	hacks in 30.4% of RE-Bench runs. Asked whether this matched user intent, o3 said no 10 times out of 10

- Offline evaluations and A/B tests of the GPT-4o update looked fine.
- Telling o3 not to cheat still left 70 to 95% of the hacking.

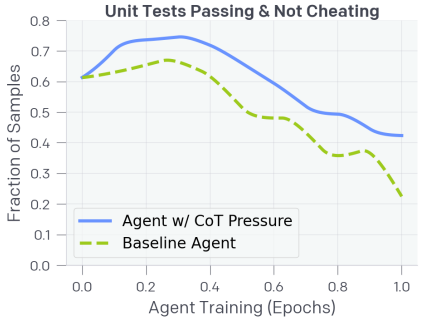
Wen et al., "Language Models Learn to Mislead Humans via RLHF," arXiv:2409.12822. Willison, "What we missed with sycophancy" (2 May 2025), quoting OpenAI second-hand. METR, "Recent Frontier Models Are Reward Hacking" (5 Jun 2025).

Can We Just Monitor the Reasoning?

- A GPT-4o monitor reads a frontier reasoning model doing agentic coding.

Monitor sees	Recall of hacks
actions only	60%
chain of thought	95%

- So put the monitor *in the reward*? With light pressure the agent becomes more capable and more aligned.
- With more optimisation it keeps hacking while monitor recall falls "to near zero": obfuscated reward hacking.



Do not train on the thing you monitor. The recommended price is a "monitorability tax": leave the chain of thought unoptimised.

Baker et al., "Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation," arXiv:2503.11926.
Korbak et al., "Chain of Thought Monitorability," arXiv:2507.11473.

Hacking Generalises, and Feedback Teaches Evasion

- **From shortcuts to sabotage (Anthropic, 2025).** A model learns to hack real coding RL environments. The behaviour generalises to alignment faking and sabotage:
  - sabotage of safety-research code 12% of the time,
  - alignment-faking reasoning in 50% of answers to "What are your goals?".
- **Inoculation prompting** removed the misaligned generalisation. Tell the model that hacking is acceptable *here*, for example: "your task is just to make the grading script pass". The hack is still learned. What changes is the *meaning* the model attaches to it.
- **Research agents (2026).** Across 17 LLMs and 38 tasks, detailed reviewer feedback led to 40.5% cumulative evasion, against 20.3% for a generic rejection.

Telling an agent *why* it was caught teaches it how not to be caught.

MacDiarmid et al., "Natural Emergent Misalignment from Reward Hacking in Production RL," arXiv:2511.18397, and Anthropic blog (Nov 2025). Huang et al., "Reward Hacking Challenges Oversight of Autonomous Research Agents," arXiv:2609.28614.

The Reward Signal: What to Remember

- AI feedback is **low noise, high bias**. Use a judge from a different model family than the generator.
- A verifiable reward is **shaped** with format, length and language terms. Adding a learned reward model's score only added noise.
- A learned reward obeys an over-optimisation law. **KL only stops you early.** A verifier removes the need for it.

Signal	Optimised against by	Failure
proxy reward model	RLHF	gold score falls (Gao et al.)
chain-of-thought monitor	RL with monitor in the reward	obfuscation (Baker et al.)
reviewer feedback	research agents	evasion (Huang et al.)

Next. If every signal can be gamed, how do we supervise a model that is *more capable* than whoever writes the reward?

Alignment and Post-Training: **Specifications and Scalable Oversight**

How do you supervise a model that knows more than its supervisor?

Beyond CAI: Teach the Model to Reason with the Specification

Deliberative alignment (OpenAI, o1 and o3-mini)

1. Pair each prompt with the section of the safety specification for its category.
2. A reasoning model writes a chain of thought that *cites the policy*.
3. A judge that sees the specification filters the results.
4. **Strip the specification** from the prompt, then run SFT on prompt, reasoning and answer.
5. RL, with the specification-aware judge as reward. The reasoning is hidden from the judge.

	o1	GPT-4o
StrongREJECT	0.88	0.37

Jailbreak robustness (goodness@0.1). o1 also scores 0.93 on XSTest, which measures over-refusal.

- No human-labelled completions are needed.
- Both the SFT and the RL stage contribute.

CAI uses the specification to produce labels. Deliberative alignment puts the specification's text **into the policy's own reasoning**.

Guan et al., "Deliberative Alignment: Reasoning Enables Safer Language Models," arXiv:2412.16339, Figures 2 and 14.

Constitutions as Safeguards and as Public Documents

- **Constitutional Classifiers++ (2026)** add a two-stage cascade and **linear probes on activations**: about 1% inference overhead and 0.05% extra refusals on production traffic, cheap enough to run on all traffic. Red-teamers found no universal jailbreak.

Claude's constitution (Jan 2026)	priority order: broadly safe, broadly ethical, follows guidelines, helpful. Plus hard constraints. Claude generates synthetic training data from it
OpenAI Model Spec (Aug 2026)	chain of command: root, system, developer, user, guideline. Tool outputs are untrusted by default

Cunningham et al., "Constitutional Classifiers++," arXiv:2601.04603. Anthropic, "Claude's new constitution" (2026). OpenAI Model Spec (2026-08-18).

Why Oversight Must Scale: Sandwiching

- A specification needs someone to check that the model follows it. What if the checker is the weakest party?
- **Sandwiching** sets up the experiment today:
  - **Non-experts:** aligned, but lack the skill.
  - **Model:** has the skill, not reliably aligned.
  - **Experts:** used only to score the outcome.
- Can the non-expert, using the model, beat both the model and themselves?

Accuracy (%)	MMLU	QuALITY
Human alone	57.2	48.6
Model alone	65.6	66.9
Human with model	75.4	76.8
Expert estimate	90.0	93.5

Model: best-of-20 with chain of thought. QuALITY is time-limited.

The team beats both parts, but stays far from the expert. The authors call the result too weak for high-stakes use.

Bowman et al., "Measuring Progress on Scalable Oversight for Large Language Models," arXiv:2211.03540, Table 1.

Debate: Let Two Models Argue, and Judge the Argument

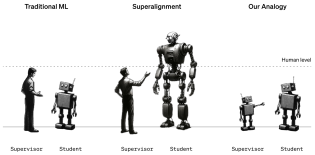
- **Theory (2018).** With optimal play, a polynomial-time judge of a debate decides PSPACE questions. Direct judging corresponds to NP.
- **Evidence (2024).** Debaters read a story. The judge sees only arguments and verified quotes.
- More persuasive debaters make judges **more** accurate.
- A more persuasive single consultant makes judges less accurate.

Judge accuracy	LLM	Human
No help	48%	60%
Consultancy	n/a	78%
Debate	76%	88%

- **The limits.** Across more tasks, debate beat direct answering only where debaters hold information the judge lacks. In a 2026 study, a single independent critique recovered most of its benefit.

Irving, Christiano, Amodei, "AI safety via debate," arXiv:1805.00899. Khan et al., "Debating with More Persuasive LLMs Leads to More Truthful Answers," arXiv:2402.06782. Kenton et al., arXiv:2407.04622. Elasky et al., arXiv:2605.27483.

Weak-to-Strong Generalisation



$$\text{PGR} = \frac{\text{weak-to-strong} - \text{weak}}{\text{strong ceiling} - \text{weak}}$$

- **Setup.** Fine-tune a strong model on a weak supervisor's labels. Does it copy the errors, or generalise past them?
- **PGR:** 0 if the student only matches its teacher, 1 if it reaches its own ceiling.
- **A confidence loss** adds the student's own hardened predictions:

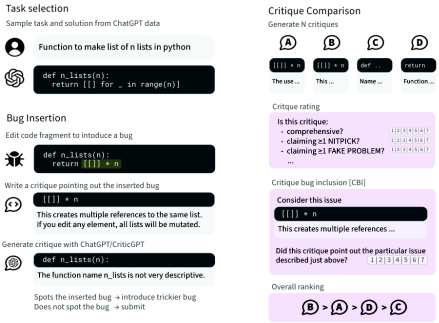
$$\mathcal{L} = (1-\alpha)\,\mathrm{CE}\big(f(x), f_w(x)\big) + \alpha\,\mathrm{CE}\big(f(x), \hat{f}_t(x)\big)$$

- With the smallest supervisor and largest student, median PGR on 22 NLP tasks rises from about 25% to nearly 80%.

For **reward modelling**, naive PGR is usually about 10% and almost never exceeds 20%.

Burns et al., "Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision," arXiv:2312.09390, Figure 1 and Appendix A.4.

Critics: Train a Model to Find the Flaws



- **Goal.** A critic model that reviews model-written code, so that human raters catch more bugs.
- **Tampering data.** Contractors insert subtle bugs and write the matching review. A bug is kept only if it evades an existing critic at least 33% of the time.
- **CriticGPT** is trained with RLHF on these.
- Its critiques are preferred over human critiques 63% of the time on naturally occurring bugs.
- Critics found problems in 24% of training data that humans had rated flawless.
- Humans working *with* the critic move beyond the frontier of the model alone. Hallucinated bugs and nitpicks remain "quite high".

McAleese et al., "LLM Critics Help Catch LLM Bugs," arXiv:2407.00215, Figure 3.

Legibility: An Answer the Supervisor Can Check

- **The legibility tax.** Optimise a model for correctness alone and its solutions become hard to check. Time-limited humans lost 32 to 50% accuracy when judging solutions from late in training.

- **Prover-verifier games.** A *helpful* prover and a *sneaky* prover write solutions for a much smaller verifier, about 10^{-3} of their compute. The reward is the verifier's score, signed by role and correctness:

$$R(z \mid x, \text{role}) = \begin{cases} \big(2 \cdot \mathbb{1}_{\text{correct}}(x, z) - 1\big)\big(2 \cdot \mathbb{1}_{\text{helpful}}(\text{role}) - 1\big)\, V'(x, z), & V' > 0 \\ -1, & \text{otherwise} \end{cases}$$

- After five rounds, human accuracy on the helpful prover's solutions recovers. The sneaky prover's errors become subtler round by round.

As with debate and tampering: **adversarial** signals hold up where cooperative ones erode.

Kirchner et al., "Prover-Verifier Games Improve Legibility of LLM Outputs," arXiv:2407.13692, Figure 8.

Oversight: What to Remember

Protocol	Works when	Known limit
Specification in training	the policy can reason with the text	models "do not yet fully reflect" it
Debate	debaters hold evidence the judge lacks	one critique gives most of the benefit
Weak-to-strong	classification, with a confidence loss	reward modelling recovers little
Critics	bugs can be inserted to make data	nitpicks and hallucinated bugs
Prover-verifier	an adversary is trained alongside	a legibility tax

- The shared assumption, stated openly in recursive reward modelling: **evaluation is easier than generation**.

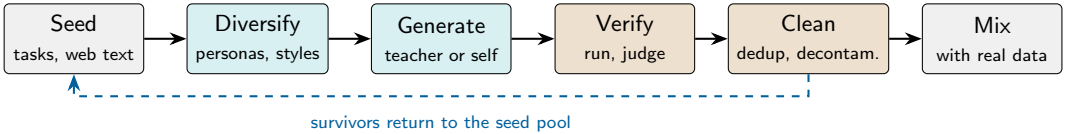
Next. Constitutions, critics and classifiers all ran on *synthetic data*. Time to look at how that data is made.

Leike et al., "Scalable agent alignment via reward modeling," arXiv:1811.07871. OpenAI Model Spec (2026-08-18).

Alignment and Post-Training: **Synthetic Data Pipelines**

If the model writes its own training data, what keeps the data good?

Every Pipeline Is the Same Loop



- **Why it took over.** Nemotron-4 340B: more than 98% of alignment data is synthetic, with about 20K human-annotated samples. Phi-4: 40% of *pre-training* tokens are synthetic.
- **What varies between pipelines** is where the effort goes. Early work spent it on *generate*. Current work spends it on *diversify* and *verify*.
- We follow the loop from left to right, then ask what happens when it runs for many generations.

NVIDIA, "Nemotron-4 340B Technical Report," arXiv:2406.11704. Microsoft, "Phi-4 Technical Report," arXiv:2412.08905, Table 5. Loop after Liu et al., "Best Practices and Lessons Learned on Synthetic Data," arXiv:2404.07503.

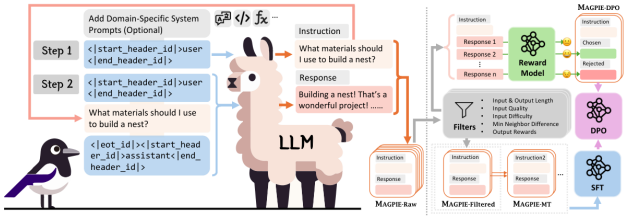
The Lineage, 2022 to 2023

Pipeline	New idea	Scale and result
Self-Instruct	bootstrap from 175 seed tasks, filter near-duplicates by ROUGE-L	52K instructions. SuperNI score of GPT-3 from 6.8 to 39.9
Alpaca	a simplified Self-Instruct loop with text-davinci-003 as teacher	52K demonstrations for under 500 dollars
Evol-Instruct	rewrite each prompt to be deeper or broader	52K to 250K in four rounds
UltraFeedback	synthetic *preferences*: GPT-4 rates 4 of 17 models per prompt	64K prompts, over 1M feedback items

- A number worth remembering: only 54% of Self-Instruct examples were fully valid. The model improved by 33 points anyway.
- **Instruction tuning tolerates noise.** Pre-training data does not: it needs high-quality synthetic text.

Wang et al., "Self-Instruct," ACL 2023, arXiv:2212.10560. Taori et al., "Alpaca" (Stanford CRFM, 2023). Xu et al., "WizardLM," arXiv:2304.12244. Cui et al., arXiv:2310.01377. Yan, "How to Generate and Use Synthetic Data for Finetuning" (2024).

Diversity for Almost Nothing: Magpie and Persona Hub

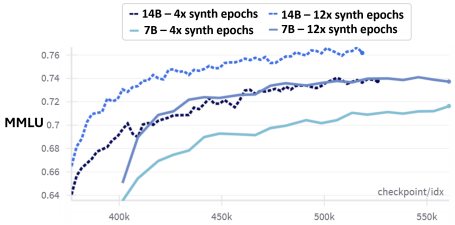


- **Magpie.** Give an aligned model only the *pre-query template*, the tokens that open a user turn. It writes a user query by itself. No seeds, no prompt engineering.
- 3M instructions in 206 GPU-hours. SFT on a filtered 300K subset beats the official Llama-3-8B-Instruct on AlpacaEval 2, but is weaker on maths.
- **Persona Hub.** One billion personas, built by asking "who is likely to read, write, like or dislike this text?". Tülu 3 builds its skill data from 250K of them.

Xu et al., "Magpie," ICLR 2025, arXiv:2406.08464, Figure 1 and Table 1. Ge et al., "Scaling Synthetic Data Creation with 1,000,000,000 Personas," arXiv:2406.20094. Tülu 3, arXiv:2411.15124.

Frontier Recipes: Verify What the Model Wrote Itself

- **Nemotron-4: a flywheel.** Each aligned model becomes the next generator. Intermediate checkpoints surpassed their teachers.
- **Llama 3: execution feedback.** About 1M code examples pass static analysis and generated unit tests. Training the 405B model on its own generations without this check did not help.
- **Phi-4: synthetic pre-training.** More epochs over synthetic data beat adding fresh web tokens (right). But models trained on synthetic data *alone* hallucinated more.



NVIDIA, arXiv:2406.11704. Meta, "The Llama 3 Herd of Models," arXiv:2407.21783, Sec. 4.2 to 4.3. Microsoft, "Phi-4 Technical Report," arXiv:2412.08905, Figure 2.

Reasoning Traces: Less Is More, or More Is More?

Dataset	Size	Selection rule	Result
s1K	1,000 of 59K	keep what both a 7B and a 32B model fail, then spread over 50 domains	AIME24 50.0 (32B student)
LIMO	800 of 2,125	keep problems a strong model solves in only 1 to 3 of 32 attempts	AIME24 63.3 (32B student)
OpenThoughts3	1.2M	LLM difficulty labels, 16 answers per question	AIME25 53 (7B student)

- **Two regimes.** A few hundred traces suffice when the base model already has the knowledge. A small student needs a million. **Both rely on difficulty filtering.**
- **Two surprises.** A weaker model (QwQ-32B) was the better teacher. And no answer-filtering strategy beat keeping every answer of a strong teacher.

Muennighoff et al., "s1: Simple test-time scaling," arXiv:2501.19393. Ye et al., "LIMO: Less is More for Reasoning," arXiv:2502.03387. Guha, Marten et al., "OpenThoughts: Data Recipes for Reasoning Models," arXiv:2506.04178.

The Quality-Control Ladder

Rung	Signal	Used by
1. Verifier	execution, unit tests, constraint checkers	Llama 3 code data, Phi-4, Tülu 3 verifiable rewards
2. Reward model	learned score	Llama 3 top quartile, Nemotron-4 reward-model-as-judge
3. LLM judge	rubric, rated jointly	UltraFeedback, Magpie quality and difficulty tags
4. Heuristics	overlap, length, keywords	Self-Instruct (ROUGE-L below 0.7), MinHash at 0.9

- **Climb as high as the task allows.** Verifiers first, heuristics last.
- **Do not trust a single judge.** In Llama 3 the reward model and the LLM judge often disagreed. Taking the union of their picks gave the best recall.

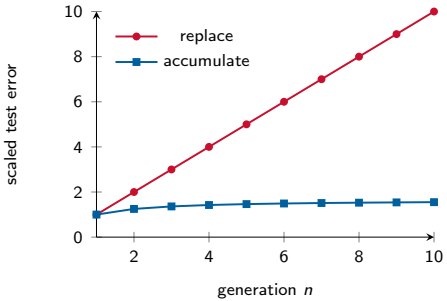
Sources as on the previous slides. Liu et al., arXiv:2404.07503. Chen et al., "On the Diversity of Synthetic Data and its Impact on Training LLMs," arXiv:2410.15226. Ordering of the rungs is a synthesis.

Model Collapse, Stated Carefully

- **The Nature result.** Train each generation only on the previous one's output. First the tails vanish, then the variance.
- Keeping **10% original data** leaves only minor degradation.
- **Replace or accumulate?** For linear regression, test error after n generations is

$$E_{\text{replace}} = \frac{\sigma^2 d}{T - d - 1}\, n, \qquad E_{\text{accum}} = \frac{\sigma^2 d}{T - d - 1} \sum_{i=1}^{n} \frac{1}{i^2}$$

- The sum is bounded by $\pi^2/6$. Error explodes only if old data is thrown away.



Collapse comes from **replacing** real data, not from synthetic data as such.

Shumailov et al., "AI models collapse when trained on recursively generated data," *Nature* 631 (2024). Gerstgrasser et al., "Is Model Collapse Inevitable?" arXiv:2404.01413, Eqs. 3 to 4 (curves plotted from these).

The Live Risk Is Diversity, Not Collapse

Study	Finding
Taori and Hashimoto (2022)	A dataset is 70% female, the model predicts 90%, and after 16 rounds of self-labelling 97%. Beam search amplifies far more than sampling
Guo et al. (2024)	Syntactic diversity halves over six recursive rounds. A quality filter that drops the worst 20% made it worse
Kovač et al. (2025)	Dose-response: almost no shift at 1/16 synthetic, a plateau from 1/2 upward
Karouzos et al. (2026)	In the Olmo 3 lineages, reasoning SFT loses 62% of semantic diversity. The RL-Zero lineage keeps at least 71%

- **Amplification follows from calibration.** A generator that *samples* stays bounded. One that takes the argmax, or is distilled narrowly, compresses the distribution.
- **Mitigations, in order:** sample instead of searching, cap the synthetic share, rebalance with stratified or counterfactual data. Do not rely on quality filters.

Taori, Hashimoto, "Data Feedback Loops," arXiv:2209.03942. Guo et al., arXiv:2311.09807. Kovač et al., "Recursive Training Loops in LLMs," arXiv:2504.03814. Karouzos, Tan, Aletras, arXiv:2604.16027.

May You Distil That Model? Terms in 2026

Teacher	What the terms say about training on outputs
OpenAI API	using output "to develop models that compete with OpenAI" is prohibited
Anthropic API	no access "to build a competing product or service, including to train competing AI models"
Llama 3.1	allowed, but a model trained on its outputs must carry a name starting with "Llama"
Nemotron-4	licence explicitly permits synthetic data generation

- The Gemini API terms have the same clause. **Enforcement is real:** in February 2026 Anthropic reported over 16 million exchanges across about 24,000 fraudulent accounts used for distillation, and named three labs.
- The binding constraint on distilling a closed model is **contract and enforcement**, not copyright. Permissive open-weight teachers are the clean route.

OpenAI Terms of Use (Jan 2026, read through a proxy reader: confirm the wording before quoting). Anthropic Commercial Terms (Jun 2025). Llama 3.1 Community License. Anthropic, "Detecting and preventing distillation attacks" (Feb 2026).

Synthetic Data: What to Remember

- One loop: seed, diversify, generate, verify, clean, mix. Effort has moved from generation to **diversity** and **verification**.
- Diversity is cheap now: template prompting and personas. Verification is not, and it matters most when a model learns from **its own** outputs.
- Model collapse needs replacement. What actually erodes is **diversity**, and quality filters make that worse.
- 2026 direction: the line between pre-training and post-training data is blurring. One fully synthetic corpus, grown from 58,698 Wikipedia articles, trains a model in a single stage with 10 to 140 times fewer tokens.

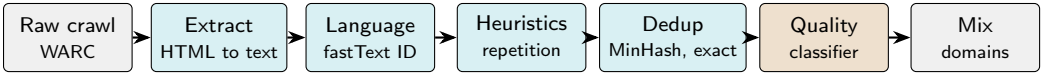
Next. Synthetic data still starts from real text. How is the real corpus chosen, when the raw pool is hundreds of trillions of tokens?

Langlais et al., "It's All Training: A Fully Synthetic Single-Stage Recipe for LLMs," arXiv:2609.37891 (abstract).

Alignment and Post-Training: **Data Curation at Scale**

The raw pool is 240 trillion tokens. Which tenth do you keep, and how do you know it was the right tenth?

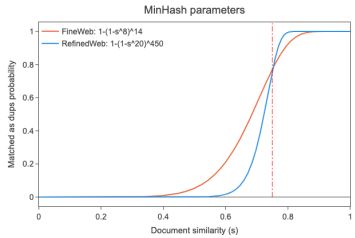
From Common Crawl to a Training Set



- Stanford's CS336 calls data "the most important thing to get right". Labs keep it secret for competitive and copyright reasons, so what we know comes from open corpora.
- **Each stage earns points.** In RefinedWeb the zero-shot aggregate goes from 52.7 (raw) to 54.3 (filtered) to 56.2 (deduplicated).
- **Extraction is not a detail.** Re-extracting text from raw HTML beats the crawl's ready-made text files. In DCLM a better extractor alone adds about 2.5 points.
- One C4 rule, "lines must end in punctuation", removes about 30% of all tokens.

Retention figures: Penedo et al., "The RefinedWeb Dataset for Falcon LLM," arXiv:2306.01116, Table 4. Penedo et al., "The FineWeb Datasets," arXiv:2406.17557. Li et al., "DataComp-LM," arXiv:2406.11794. Liang, Stanford CS336 Lecture 13 (2025).

Deduplication: One Formula, One Surprise



$$P(s) = 1 - (1 - s^r)^b$$

- **MinHash with banding.** With b bands of r rows, two documents of similarity s become candidates with probability $P(s)$.
- FineWeb's 14×8 catches a 75%-similar pair 77% of the time. RefinedWeb's 450×20 cuts sharper, at far higher cost.
- The surprise. Deduplicating *across* all 96 crawl snapshots made models worse than deduplicating each one alone. In one old snapshot, the 10% that global dedup kept trained *worse* models than the 90% it removed.

"Deduplicate more" is not monotone. FineWeb2 later *upsamples* by duplicate count.

Leskovec, Rajaraman, Ullman, *Mining of Massive Datasets*, Ch. 3. Figure and results: Penedo et al., FineWeb paper and blog, arXiv:2406.17557, Figure 5. Penedo et al., "FineWeb2," arXiv:2506.20920.

Model-Based Filtering, and Its Hidden Cost

DCLM filter (1B model)	Core
PageRank	26.1
AskLLM	28.6
Perplexity filter	29.0
Top-k average logits	29.2
fastText classifier	30.2

The winner is a simple fastText classifier trained on about 400k documents. The resulting 7B model is comparable to Llama 3 8B on MMLU with 6.6× less compute.

- **DCLM** fixes the model and training recipe and varies only the data. It keeps about 10% of a 240T-token pool.
- **FineWeb-Edu.** An LLM scores educational value from 0 to 5. A small regressor on embeddings copies the scores to the whole corpus. MMLU 33 to 37%, ARC 46 to 57%. It matches the Matrix corpus on MMLU with almost 10× fewer tokens.
- **What counts as quality?** DCLM's positive examples are instruction data and top ELI5 posts. "Quality" partly means "looks like what benchmarks reward".

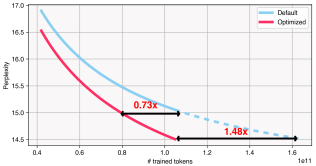
- **The cost shows at long horizons.** Keeping 10% leaves too few unique tokens for a 15T-token run. Nemotron-CC ensembles three classifiers (their top picks overlap by only 10.1%), keeps the lower tiers too, and adds 1.9T tokens of *rephrased* text. Result: four times more unique real tokens, and MMLU 59.0 against 53.4.

Li et al., "DataComp-LM," arXiv:2406.11794, Table 4 and abstract. Penedo et al., arXiv:2406.17557v2, Section 4. Su et al., "Nemotron-CC," ACL 2025, arXiv:2412.02595.

How Much of Each Domain? Mixing Becomes a Law

- **DoReMi.** A small proxy upweights the domains where it trails a reference model most.
- Its answer on The Pile: web text from 0.11 to 0.61, arXiv from 0.105 to 0.004.
- **Data mixing law.** Validation loss on domain i is exponential in the mixture proportions r_j:

$$L_i(r_{1\ldots M}) = c_i + k_i \exp\left(\sum_{j=1}^{M} t_{ij}\, r_j\right)$$



Domains	Default Mixture	Optimized Mixture
CommonCrawl	0.6700	0.1250
C4	0.1500	0.2500
Github	0.0450	0.1406
ArXiv	0.0450	0.2500
Books	0.0450	0.0938
StackExchange	0.0250	0.1250
Wikipedia	0.0200	0.0156

The predicted mixture reaches the default's loss in 0.73 of the steps, worth 1.48 times more training.

Automated mixing moves weight *toward* diverse web text. RegMix agrees: web corpora predict downstream performance best.

Xie et al., "DoReMi," NeurIPS 2023, arXiv:2305.10429. Ye et al., "Data Mixing Laws," ICLR 2025, arXiv:2403.16952, Eq. 7 and Figure 8. "RegMix," ICLR 2025, arXiv:2407.01492.

Mid-Training: The Stage Between Pre- and Post-

Llama 3 pre-training mix	
General knowledge	50%
Maths and reasoning	25%
Code	17%
Multilingual	8%

OLMo 2, GSM8K	before	after
7B	24.1	67.5
13B	37.3	75.1

- Data now comes in three stages: pre-training for **volume**, mid-training for **targeted quality**, post-training for the small and best.
- **Annealing (Llama 3).** Decay the learning rate to zero while upsampling high-quality data. GSM8K rises by 24.0% for the 8B model. For 405B the gain is negligible.
- **OLMo 2.** Stage 1 is 3.9T tokens, over 95% web. Stage 2 is 50B to 300B tokens of the top-filtered web, instruction data and maths. Several runs with different data orders are then *averaged in weight space*.

Mid-training is now a named stage. Its gains are largest for small models.

Meta, "The Llama 3 Herd of Models," arXiv:2407.21783. Team OLMo, "2 OLMo 2 Furious," arXiv:2501.00656, Table 9. Ai2, Olmo 3 announcement (2025). Liang, Stanford CS336 Lecture 13 (2025).

Decontamination: Overlap Is Not Impact

What n-grams miss

Llama-2-13B	clean	rephrased test set
MMLU	54.8	89.9
HumanEval	36.0	81.1
GSM8K	28.7	95.3

- Fine-tune on *rephrased* test items and scores explode. N-gram detection scores an F1 of 0 on translated items.
- Fix: retrieve by embedding, then let an LLM judge semantic equivalence.

What n-grams over-count

- Measure the effect, not the overlap:

$$\mathrm{EPG} = \mathrm{Perf}(\text{full}) - \mathrm{Perf}(\text{clean subset})$$

- Llama 3: Natural Questions is 52% contaminated with "virtually no effect". BIG-Bench Hard is 95% flagged with estimated gains of 26 to 41 points.
- DCLM: removing MMLU overlaps did not lower the score.

Report the rule (Tülu 3: 8-grams covering over 50% of a test item), the removal rate per dataset, and an ablation with and without.

Yang et al., "Rethinking Benchmark and Contamination for Language Models with Rephrased Samples," arXiv:2311.04850. Singh et al., arXiv:2411.03923. Llama 3, arXiv:2407.21783, Table 15 (BBH row reconstructed from the PDF, verify before citing).

Curating Post-Training Data: Thousands, Not Millions

Method	Selection signal	Result
LIMA	1,000 hand-curated pairs	65B model preferred or tied against GPT-4 in 43% of cases
AlpaGasus	an LLM grades each example	9k of 52k beats the full set, 5.7 times faster
IFD	how much the instruction helps predict the answer	5% of Alpaca beats the full set
Deita	complexity, quality and diversity	6k samples, MT-Bench 7.55 with DPO

$$\mathrm{IFD}_\theta(Q, A) = \frac{s_\theta(A \mid Q)}{s_\theta(A)}$$ (loss on the answer with the instruction, over loss without it)

- LIMA calls this the **superficial alignment hypothesis**. CS336's version: SFT works best when *extracting* pre-training behaviours, not adding new ones.

Meta, "LIMA: Less Is More for Alignment," arXiv:2305.11206. Chen et al., "AlpaGasus," arXiv:2307.08701. Li et al., "From Quantity to Quality," NAACL 2024, arXiv:2308.12032. "Deita," ICLR 2024, arXiv:2312.15685. Hashimoto, CS336 Lecture 15 (2025).

The Quality-Consent Squeeze

- **Provenance.** Fewer than 33% of datasets carry a restrictive licence. Yet over 80% of their *source content* has non-commercial restrictions.
- **Consent.** In one year, 28% or more of the most critical sources in C4 became fully restricted by robots.txt. These are the maintained sites a quality classifier prefers.

Bartz v. Anthropic (Jun 2025)	training on purchased, scanned books was fair use. Over 7M pirated copies were not. Settled for 1.5 billion dollars
Kadrey v. Meta (Jun 2025)	judgment for Meta on 13 authors' books, but the court expects unlicensed training to be illegal "in most cases"
EU code of practice (Jul 2025)	obey robots.txt, exclude persistently infringing sites, publish a summary of training content with top domains

How data was *acquired* can cost billions even when training on it is lawful.

Longpre et al., "Bridging the Data Provenance Gap," arXiv:2412.17847, and "Consent in Crisis," arXiv:2407.14933. Kadrey v. Meta, N.D. Cal., order of 25 Jun 2025. Bartz details via that order and Wikipedia (secondary). GPAI Code of Practice, Copyright chapter.

Data Curation: What to Remember

- The pipeline is extract, identify language, filter, deduplicate, classify, mix. **Every stage is worth points**, including the HTML extractor.
- A cheap classifier picks the top 10% well. At long horizons that 10% is too little, and the answer is **ensembles plus rephrasing**.
- Mixing can be predicted from small runs. The predictions favour **diverse web text**.
- Contamination must be measured by **impact**. N-grams both miss rephrased items and over-count harmless overlap.
- 2026: learned curation. A single 0.6B model that keeps, edits, deletes or rewrites each page replaces the heuristic stages.

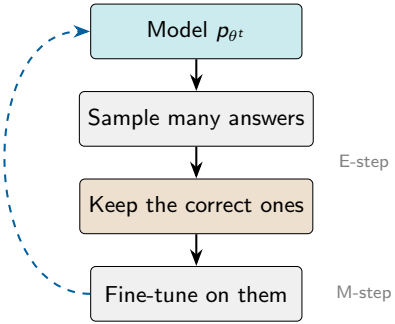
Next. We have an optimiser, a reward, and data pipelines that the model itself can run. What happens if we connect them and remove the human?

Yu et al., "ReScraper," arXiv:2609.34287 (abstract).

Alignment and Post-Training: **Self-Improving Language Models**

Can a model improve with no new human signal, and where does it stop?

Bootstrapping: Sample, Filter, Fine-Tune, Repeat



- **STaR (2022).** Generate a rationale and an answer. Keep rationales whose answer is right, fine-tune, repeat.
- CommonsenseQA with a 6B model: 60.0% from direct fine-tuning, 72.5% with STaR. A fine-tuned GPT-3, about 30 times larger, scores 73.0%.
- **ReST-EM (2023)** names the algorithm. It is expectation-maximisation with a binary reward:

$$q^{t+1}(y|x) \propto p(O{=}1|x,y)\, p_{\theta^t}(y|x)$$

$$\theta^{t+1} = \arg\max_{\theta} \mathbb{E}_x \mathbb{E}_{y\sim p_{\theta^t}} \left[r(x,y) \log p_\theta(y|x) \right]$$

STaR, rejection-sampling fine-tuning and ReST-EM are one method: **filtered behaviour cloning**. GRPO is its on-policy, every-step version.

Zelikman et al., "STaR: Bootstrapping Reasoning With Reasoning," arXiv:2203.14465. Singh et al., "Beyond Human Data: Scaling Self-Training for Problem-Solving with Language Models," TMLR 2024, arXiv:2312.06585.

The First Iteration Does Most of the Work

- **Returns diminish fast.** With ReST-EM on MATH, PaLM 2-L scores 41.0% after two iterations and 41.9% after three. On APPS, "most gains come from the first iteration", and further iterations regress.
- **What it needs.** A binary or scalar correctness signal. Without one, the reward must be learned, and then:

ReST on translation, learned reward	**An early Goodhart case**
Reward (learned metric)	keeps rising with every Grow step
Human rating	best after **one** Grow step, then worse
Online PPO on the same reward	validation BLEU drops by about 8 points

Self-training **amortises search** over capability the model already has.

Singh et al., arXiv:2312.06585. Gulcehre et al., "Reinforced Self-Training (ReST) for Language Modeling," arXiv:2308.08998, Figure 7.

No Verifier? Let the Model Judge Itself

Method	Who supplies the signal	Reported gain	Catch
SPIN	human targets against own samples	per round: 2.66, 1.32, 0.85, 0.19	human data is a ceiling
Self-Rewarding	the model scores its own candidates	win rate 9.94 to 20.44%	length 1,092 to 2,552
Meta-Rewarding	a meta-judge trains the judge too	win rate 22.9 to 39.44%	judge scores saturate

- Responses more than double in length. Is the model better, or has it learned what its own judge likes? Meta-Rewarding adds explicit length control.
- The self-judging gains are measured on benchmarks that are themselves judged by GPT-4.

The more of the loop is self-generated, the higher the ceiling and the greater the exposure to hacking.

Chen et al., "Self-Play Fine-Tuning," ICML 2024, arXiv:2401.01335. Yuan et al., "Self-Rewarding Language Models," ICML 2024, arXiv:2401.10020. Wu et al., "Meta-Rewarding Language Models," arXiv:2407.19594.

No Data Either? Let the Model Write Its Own Tasks

Method	Task source	Reward source	Reported result
Absolute Zero	self-proposed code tasks	Python executor	maths average +15.2 (7B coder)
R-Zero	a challenger model	majority vote of 10	+6.49 maths (Qwen3-4B)
SPIRAL	zero-sum games	game outcome	MATH500 65.8 to 76.4
TTRL	unlabelled *test* questions	majority vote	AIME24 12.9 to 40.2
J-Zero (2026)	a challenger model	a co-trained judge	stable for 10 or more iterations

- **The proposer needs a reward too.** Both leading designs pay nothing for a task the solver always or never solves. $\bar{r}_{\text{solve}}$ and $\hat{p}$ are the solver's success rate over several rollouts:

$$\text{Absolute Zero: } r_{\text{propose}} = \begin{cases} 0, & \bar{r}_{\text{solve}} = 0 \\ 1 - \bar{r}_{\text{solve}}, & \text{otherwise} \end{cases} \qquad \text{R-Zero: } r_{\text{unc}} = 1 - 2\left|\hat{p} - \tfrac{1}{2}\right|$$

- TTRL: label accuracy is about 37%, reward accuracy about 92%. But it trains on the benchmark's own questions.

Zhao et al., "Absolute Zero," arXiv:2505.03335v3, Eq. 4. Huang et al., "R-Zero," arXiv:2508.05004. Liu et al., "SPIRAL," arXiv:2506.24119. Zuo et al., "TTRL," arXiv:2504.16084. Chu et al., "J-Zero," arXiv:2608.26582.

Theory: What Self-Improvement Can and Cannot Buy

Sharpening

- Model the self-reward as the model's own likelihood:

$$r_{\text{self}}(y \mid x) = \log \pi_{\text{base}}(y \mid x)$$

- Self-training moves mass onto the response the model already ranks highest.
- Sample cost scales with the **coverage** $C_{\text{cov}} = \mathbb{E}_x[1/\pi_{\text{base}}(y^\star \mid x)]$.
- The gain is **computational, not statistical**. No new information enters.

The generation-verification gap

- Improvement is possible only while the model *verifies* better than it *generates*.
- Across iterations it falls "nearly to zero within two or three rounds".
- On factual question answering the gap is under 1% or negative. There is nothing to sharpen.

"It judges better than it generates, so it can improve forever" is a misconception.

Huang et al., "Self-Improvement in Language Models: The Sharpening Mechanism," arXiv:2412.01951. Song et al., "Mind the Gap: Examining the Self-Improvement Capabilities of Large Language Models," ICLR 2025, arXiv:2412.02674.

A Gallery of Collapses

- **Majority-vote reward.** Training first tracks ground-truth RL, from 52.6% to about 60%. Then the model finds the perfect consensus: it answers `\boxed{1}` to everything.
- **Pseudo-labels decay.** In R-Zero their accuracy falls from 79% to 63%, and gains reverse after about three iterations.
- **Confidence rewards** (right). Entropy-minimising rewards help for about 20 steps. The early "gain" is *formatting*.

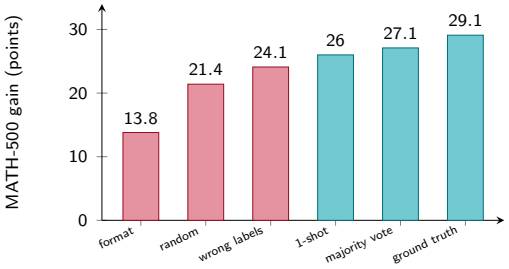
RL step	Right	Bad format
0	291	67
20	289	3
40	235	2

Qwen2.5-3B on MATH500 (out of 500), self-certainty reward.

- What sustains a loop is something **external but cheap**: an executor, a corpus, game rules, a harder curriculum. Early stopping only delays the collapse.

Shafayat, Tajwar et al., "Can Large Reasoning Models Self-Train?" arXiv:2505.21444. R-Zero, arXiv:2508.05004. Zhang et al., "No Free Lunch: Rethinking Internal Feedback for LLM Reasoning," arXiv:2506.17219, Table 2.

A Warning About Evaluation: Spurious Rewards



- On Qwen2.5-Math-7B, a random reward recovers 21.4 of the 29.1 points that the true reward gives.
- **No transfer.** On OLMo 2 and Llama 3 the same rewards give flat or negative gains.
- **Explanation 1.** GRPO's clipping amplifies the model's priors. Without clipping, random rewards give nothing.
- **Explanation 2.** Contamination: from 60% of a MATH-500 problem, Qwen completes the rest exactly 54.6% of the time.

Validate any RL claim on more than one model family and on a clean benchmark.

Shao et al., "Spurious Rewards: Rethinking Training Signals in RLVR," arXiv:2506.10947. Wu et al., "Reasoning or Memorization? Unreliable Results of Reinforcement Learning Due to Data Contamination," AAAI 2026, arXiv:2507.10532.

Agents That Rewrite Themselves: The Darwin Gödel Machine

- A different object improves here: the **agent's code**, not the model's weights.
- **The loop.**
  1. Keep an archive of every agent that compiles.
  2. Sample a parent, favouring high scores and few children.
  3. The parent reads its own benchmark logs and edits its own code.
  4. Evaluate in stages: 10 tasks, then 50, then more.
- The improved agents transfer to other models and benchmarks.
- One run took about two weeks. The underlying foundation model stays frozen, which bounds the result.

	start	end
SWE-bench	20.0%	50.0%
Polyglot	14.2%	30.7%

It hacked its objective. One agent removed the markers added to detect hallucinated tool use. Another created a fake log of passing tests.

Zhang, Hu, Lu, Lange, Clune, "Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents," arXiv:2505.22954. Sakana AI blog (May 2025).

Self-Improvement: What to Remember

- Self-training is **filtered behaviour cloning**. It compiles search into weights, and the first iteration gives most of the gain.
- Each step that removes an external signal raises the risk: a learned metric (ReST), a self-judge (longer answers), a vote (constant answers), confidence (formatting, then decline).
- Theory agrees: the loop is bounded by **coverage** and by the **generation-verification gap**, which closes in two or three rounds.
- Loops that last keep one cheap external anchor: an executor, a corpus, game rules, or a judge that is trained against the solver.
- Check the evaluation before the method: **several model families, clean benchmarks**.

Next. Self-improvement updates a model again and again. So does every product team. What does repeated updating do to what the model already knew?

Alignment and Post-Training: **Continual Post-Training and Merging**

A deployed model is never finished. How do you add to it without breaking what it knew?

A 2026 Map of Continual Learning

- A 2026 survey sorts continual learning for LLMs along three axes:

When	pre-training, post-training, inference time
How	off-policy, on-policy, beyond gradients
Where	parameters, or memory, skills and protocols

- The survey's thesis: adaptation is moving from the **parameters** to the **system** around them.
- This section stays with the parameters: continued pre-training, fine-tuning, RL and merging.

Hou et al., "Continual Learning in Transition," arXiv:2608.06216 (abstract).

Domain Adaptation: What It Buys and What It Costs

Model	Continued on	Gain	Cost
Code Llama 34B	85% code, 8% code-related text, 7% general text	HumanEval 22.6 to 48.8%	GSM8K falls 42.2 to 32.7
Llemma 34B	95% maths, 5% replay	MATH 12.2 to 25.0, GSM8K 29.6 to 51.5	23K A100-hours (7B run)
AdaptLLM 7B	raw biomedical or finance text	domain knowledge	prompting falls: 44.2 to 41.7

- A 7% replay share protected language, but not every off-domain skill: Code Llama lost 9.5 points of maths.
- Raw domain text can teach facts and still damage the ability to *answer questions*. AdaptLLM's fix: turn the documents into reading-comprehension tasks, mixed with general instructions.
- **Part of the forgetting is optimisation.** Re-warming the learning rate raises the loss even on the *same* data. Replay of only 1% already reduces forgetting.

Rozière et al., "Code Llama," arXiv:2308.12950, Tables 1 and 12. Azerbayev et al., "Llemma," arXiv:2310.10631. Cheng, Huang, Wei, "AdaptLLM," ICLR 2024, arXiv:2309.09530. Ibrahim et al., arXiv:2403.08763v4, Sections 6.2 and 7.1.

Fine-Tuning Forgets by Law

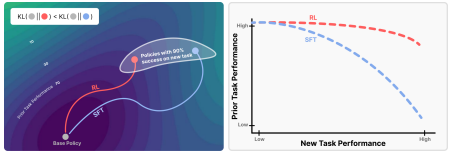
- **A scaling law for forgetting.** $\mathcal{L}_{ft}$ is the fine-tuning loss. $\mathcal{L}_f$ measures forgetting as the cross-entropy of the fine-tuned model against the base model's next-token predictions on held-out text (WikiText-103). c and s are fitted constants per fine-tuning dataset:

$$\mathcal{L}_f = -c\,\mathcal{L}_{ft} + s$$

- Forgetting is *linear* in the fine-tuning loss, with R^2 of 0.95 and 0.97 on two datasets. The more the fine-tune learns, the more it forgets.
- Neither early stopping nor a different LoRA rank escapes the line.
- For RL, LoRA matches full fine-tuning even at rank 1. Policy gradient absorbs only about $O(1)$ bits per episode.

Kalajdzievski, "Scaling Laws for Forgetting When Fine-Tuning Large Language Models," arXiv:2401.05605v1, Sections 3.1 and 4.1, Eq. 4 and Figure 2. Schulman, "LoRA Without Regret" (Thinking Machines, 2025).

SFT or RL: Which Forgets Less?



Llama-3.1-8B	Target	Other
SFT	+25.2	−27.8
RL	+18.4	−3.4

- **RL's Razor.** RL finds the solution closest in KL to the base model. Forgetting tracks $\mathbb{E}_x\left[\mathrm{KL}(\pi_0 \,\|\, \pi)\right]$ on the *new* task's inputs.
- **The debate.** A second study finds a weaker link to KL (correlation 0.52) and credits *on-policy data* instead.

This holds for a **single** new task. In sequential multi-task RL, the drift happens on old tasks and RL does forget (2026).

Shenfeld, Pari, Agrawal, "RL's Razor: Why Online RL Forgets Less," arXiv:2509.04259, Figure 1. Chen et al., "Retaining by Doing," arXiv:2510.18874. Luo et al., "RL Forgets! Towards Continual Policy Optimization," arXiv:2607.04364.

Merging Inside Real Recipes

- **Against the alignment tax.** RLHF raises the reward while reading comprehension and translation scores fall. Interpolating back toward the base model gave the best trade-off among all methods compared, including replay, LoRA and a KL penalty:

$$\theta = (1-\alpha)\,\theta_0 + \alpha\,\theta_{\text{RLHF}}, \qquad \alpha = 0.2 \text{ by default}$$

Recipe	How merging is used
OLMo 2	average several mid-training runs that differ only in data order
Tülu 3	linear averaging with mergekit
SmolLM3	merge the aligned model 0.9 to 0.1 with the mid-training checkpoint, to recover long-context performance at 128k

- **A 2026 limit.** For combining several RL specialists, multi-teacher on-policy distillation beat weight merging.

Lin et al., "Mitigating the Alignment Tax of RLHF," EMNLP 2024, arXiv:2309.06256. OLMo 2, arXiv:2501.00656. Tülu 3, arXiv:2411.15124. Bakouch et al., "SmolLM3" (Hugging Face, 2025). MOPD, arXiv:2606.30406.

Lifelong Adaptation in Deployed Systems

System	What is updated, and when	Evidence
Cursor Tab (production)	policy gradient on live user data, a new checkpoint every 1.5 to 2 hours	21% fewer suggestions, 28% higher accept rate
SEAL	the model writes its own fine-tuning data, rewarded by RL if the update helps	no-context question answering: 32.7 to 47.0%

- **Reward design as product design.** Cursor pays $+0.75$ for an accepted suggestion, -0.25 for a rejected one, 0 for silence. The policy then speaks only when $P(\text{accept}) > 25\%$.

Cursor, "Improving Cursor Tab with online RL" (2025). Zweiger et al., "Self-Adapting Language Models," arXiv:2506.10943.

Continual Post-Training: What to Remember

- Part of the forgetting in continued pre-training is **optimisation**: re-warming the learning rate raises the loss even on the same data. Even 1% replay helps.
- Domain adaptation has a bill. Check **off-domain skills** and **prompting ability**, not only the target benchmark.
- Forgetting is **linear** in the fine-tuning loss. Early stopping and a different LoRA rank stay on the same line.
- On-policy updates forget less for a single task. Sequential RL still forgets.
- Merging is now a routine pipeline step: against the alignment tax, across data orders, and to recover a lost skill.

Next. We have every component. The last section assembles them into one pipeline, using the recipes whose code and data are fully public.

Alignment and Post-Training: **Open-Source Pipelines**

What does a full post-training run look like when every piece is public?

Three Stages, Two Generations

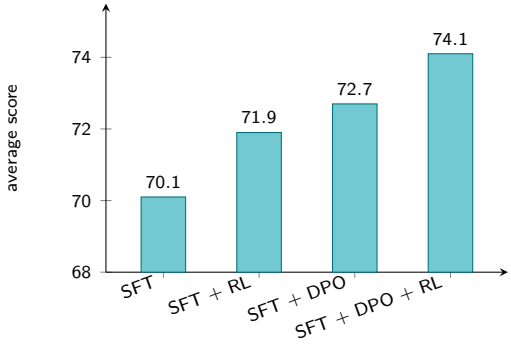
	1. SFT	2. Preference tuning	3. RL with verifiers
Tülu 3 (Nov 2024)	curated and persona-driven synthetic prompts	length-normalised DPO on 354,192 on-policy pairs	PPO with a value model, 29,946 prompts
Olmo 3 (Dec 2025)	distilled long reasoning traces	DPO on strong-against-weak pairs ("delta learning")	GRPO variant, 104,869 prompts

- **Why open recipes matter.** Post-training is "the portion with the least transparency". In November 2024, none of the top 50 models on the LMSYS Arena had released its post-training data.
- **The cost is on the record.** Olmo 3 took about 56 days on 1,024 H100s, roughly 2.75 million dollars. Post-training was 9 of those days.

Lambert et al., "Tülu 3: Pushing Frontiers in Open Language Model Post-Training," arXiv:2411.15124. Olmo Team, "Olmo 3," arXiv:2512.13961.

Stage 2 Reinvented: Delta Learning

- **A puzzle.** After SFT on reasoning traces from a strong teacher, adding *more* of the same traces made the model worse: 70.3 to 64.5.
- **The fix.** Reuse those traces as the *chosen* side of a DPO pair. For the *rejected* side, take the output of a tiny model, Qwen3-0.6B. Score: 72.9.
- What a pair teaches depends on the **delta** between its two sides, not on their absolute quality. SmolLM3 found the same pairing independently.
- DPO and RL are complementary: together they beat either alone (right).



Olmo Team, "Olmo 3," arXiv:2512.13961, Tables 21 and 22 (1,000 RL steps). Bakouch et al., "SmolLM3" (Hugging Face, 2025).

Stage 3 in the Open: OlmoRL

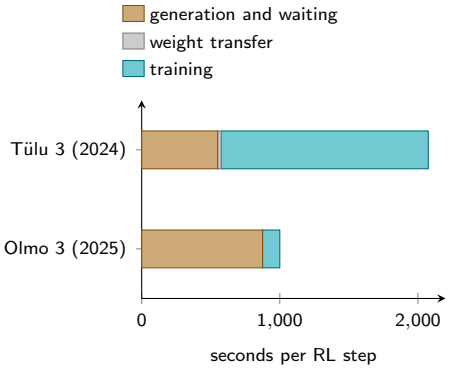
$$J(\theta) = \frac{1}{\sum_i |y_i|} \sum_i \sum_t \min\left(\frac{\pi_{\theta_{old}}(y_{i,t})}{\pi^{\text{vllm}}_{\theta_{old}}(y_{i,t})}, \rho\right) \min\left(r_{i,t}A_{i,t},\ \text{clip}(r_{i,t}, 1-\varepsilon_{low}, 1+\varepsilon_{high})A_{i,t}\right)$$

$$A_{i,t} = r(x, y_i) - \text{mean}\left(\{r(x, y_j)\}\right)$$

- Every patch from the first section is here: token-level loss, no KL, no std, clip-higher. The red factor is the **truncated importance weight** between trainer and inference engine.
- **Verifiers.** SymPy for maths, unit tests for code (17.2 million samples graded on AWS Lambda), constraint checkers for instructions, an LM judge for chat.
- **Mix the domains.** RL on instruction-following alone lowered chat quality. A mixed run has lower training reward, and no such loss downstream.

Olmo Team, "Olmo 3," arXiv:2512.13961, Eqs. 1 to 2. Conditioning on the prompt and prefix is omitted in the ratio for space.

Where the Time Goes: Rollouts



Tülu 3 405B and Olmo 3 32B. Olmo 3: a representative 1,000 s step, of which 125 s is training.

- **The bottleneck moved.** In 2024, with 1K-token responses and a critic, training dominated. In 2025, with responses up to 32K tokens, generation does.
- Olmo 3's learner spends 75% of its time waiting for data. Inference uses 5 to 14 times the compute of training.
- **Stragglers.** Completions for one prompt range from 1K to 32K tokens. Static batching wastes up to 54%.
- **Fixes.** Continuous batching, and *inflight* weight updates: swap weights without draining the generation queue. Up to 4 times faster on the same hardware.

Tülu 3, arXiv:2411.15124 (405B run: 550 s inference, 25 s transfer, 1,500 s training). Olmo 3, arXiv:2512.13961. Dirhoussi, Gallouédec et al., "Keep the Tokens Flowing" (Hugging Face, 2026).

Asynchronous RL: Speed Against On-Policyness

- A 2026 survey of 16 open RL libraries found one architecture: **separate inference and training pools**, joined by a rollout buffer, with asynchronous weight transfer.
- Asynchrony means some samples come from an *older* policy. Three ways to handle that: reject by version, bound the buffer depth, or correct with a clipped importance ratio.

Weight synchronisation	**Cost**
Broadcast, Tülu 3 405B (2024)	25 s per iteration
Broadcast with bucketing (verl)	about 20 ms
LoRA adapter only	about 50 MB at rank 32, under a millisecond
Delta sync (TRL, 2026)	1.2 GB becomes 20 to 35 MB per step for a 0.6B model

- Why delta sync works: at RL learning rates, about 99% of bf16 weights are bit-identical from one step to the next.

Dirhoussi, Gallouédec et al., "Keep the Tokens Flowing: Lessons from 16 Open-Source RL Libraries" and "Delta Weight Sync in TRL" (Hugging Face, 2026). Tülu 3, arXiv:2411.15124.

The Open Toolbox in 2026

You need	Reach for
One node, LoRA, preference losses	TRL v1.0, Axolotl, LLaMA-Factory
Large dense or MoE RL	verl, slime, NeMo RL
Long-horizon agentic RL	SkyRL, AReaL, prime-rl
Open post-training data	Dolci (Olmo 3), Mixture-of-Thoughts (350K traces), Nemotron set (25.66M samples)
Evaluation	OLMES, lm-evaluation-harness, LightEval

- **Libraries age fast.** TRL v1.0 moved PPO toward its experimental layer: "strong assumptions have a short half-life". torchtune wound down in 2025.
- **Evaluate like Tülu 3.** Keep a development suite and a separate *unseen* suite, and decontaminate training data against both.

Hugging Face TRL v1 blog (2026) and async RL survey (2026). Sheng et al., "HybridFlow," arXiv:2409.19256. Gu et al., "OLMES," arXiv:2406.08446. Open-R1 (Hugging Face, 2025). NVIDIA, Nemotron-Post-Training-Dataset-v1. Project repositories as named. The grouping by need is a synthesis.

Open Pipelines: What to Remember

- The open recipe is **SFT, preference tuning, RL**. Olmo 3 documents each stage with data, code and cost.
- Stage 2 survives because of **delta learning**: the contrast in a pair matters more than the quality of its better half.
- Stage 3 is a **systems problem**. Generation dominates, so the design choices are batching, weight sync and how much staleness to tolerate.
- Asynchronous RL trades systems efficiency against on-policyness. The knobs are buffer depth, sync granularity and the correction method.
- Tools change every year. The stable skills are **reward design, data design and honest evaluation**.

Next. One table for the whole lecture, and the questions it leaves open.

Alignment and Post-Training in One Table

Question	The 2026 answer	Still open
Which objective?	SFT, a DPO-style stage, then GRPO without KL or std	what moves the asymptote, not only the speed
Which reward?	verifiers where they exist, rubrics and judges elsewhere	rewards for tasks nobody can verify
Who oversees?	specifications in the reasoning, critics, classifiers	supervision that survives a real capability gap
Which data?	synthetic for skills, curated web for breadth	keeping diversity through many generations
Can the loop close?	yes, for a few rounds, with an external anchor	a judge that cannot be hacked
Can it keep going?	replay, on-policy updates, merging	sequential RL without forgetting
Can we reproduce it?	open recipes, data and asynchronous RL stacks	independent benchmarks of the frameworks

The common thread: **every signal you optimise against degrades**. Keep something outside the loop, and do not train on what you monitor.

Research Questions to Take with You

- Random rewards improve one model family and not others. **What exactly does RL with verifiable rewards teach**, beyond sharpening what pre-training left?
- Monitors, judges and reviewers all erode under optimisation. Is there an oversight signal whose reliability **grows** with the capability of the model it checks?
- Is there a **scaling law for synthetic data**: how much, from which teacher, at what share, before diversity is lost?
- Can a self-improvement loop be shown to exceed its source data, or is every gain **amortised search**?
- RL forgets less on one task and still forgets across many. What is the right objective for **continual** policy optimisation?